Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 09

Legarea interfeței React de API

Recenziile ajung în pagina produsului: le citim, adăugăm și ștergem din React, prin API-ul scris în laboratorul 08.

Tema	Temele 7 și 11 din planul disciplinei (examene.pe.ub.ro), aplicate: interfața React și serviciile web
Durata	2 ore
Aveți nevoie de	API-ul de recenzii din laboratorul 08 și pagina `ProductDetailsPage`
La final aveți	componenta `ReviewSection` pe pagina produsului și un commit pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- scrie funcții `fetch` pentru GET, POST și DELETE și trata erorile primite de la server;
- construi o componentă React cu stări de încărcare și eroare, care își actualizează lista fără reîncărcarea paginii;
- urmări drumul unei cereri prin aplicație și explica, pe scurt, ce este CORS.

2. Pe scurt: drumul unei cereri

Recenziile unui produs parcurg tot „lanțul” aplicației [1]:

Pas	Unde	Ce se întâmplă
1	browser, React (5173)	`getReviews(1)` face `fetch` la `…:8080/api/products/1/reviews`
2	`daw-nginx` (8080)	serverul web primește cererea și o predă lui PHP
3	`daw-php` (Symfony)	router-ul alege `ReviewController::byProduct()`; Doctrine cere datele
4	`daw-postgres`	rulează `SELECT … WHERE product_id = 1` și întoarce rândurile
5	drumul înapoi	obiectele `Review` devin JSON (`toArray()`) și ajung înapoi în browser
6	React	`setReviews(...)` schimbă starea, iar React redesenează lista [3]

CORS (*Cross-Origin Resource Sharing*). Pagina vine de la `localhost:5173`, API-ul de la `localhost:8080`: sunt **origini diferite**, iar browserul lasă JavaScript-ul să citească răspunsul doar dacă serverul confirmă, prin antetul `Access-Control-Allow-Origin`, că acceptă originea paginii [4]; în template, prin variabila `CORS_ALLOW_ORIGIN` din `backend/app/.env`. Pentru POST cu JSON și DELETE, browserul trimite întâi o cerere de verificare `OPTIONS` (*preflight*).

3. Pornirea

1. Porniți **Docker Desktop** (mesajul **Engine running**). Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`; pentru probleme, vedeți metodologia [5].

2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                 # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

4. Funcțiile de acces la API

Unde. `frontend/app/src/api.js`, **la sfârșitul fișierului**. Funcțiile noi urmează tiparul celor existente (`getProducts()`, `createProduct()`…), pe care nu le modificați.

```
// --- Recenzii (laboratorul 09) ---
// Adresa backend-ului fără „/api” la final, oricum ar fi scrisă în .env.
const REVIEWS_BASE = (import.meta.env.VITE_API_URL || 'http://localhost:8080')
  .replace(/\/+$/, '')
  .replace(/\/api$/, '');

// Transformă un răspuns cu eroare într-un obiect Error cu mesaj citibil.
async function reviewError(res) {
  const body = await res.json().catch(() => ({}));
  const fields = body.errors || {};
  const message = Object.values(fields).join(' ')
    || body.error || `Eroare ${res.status}`;
  const error = new Error(message);
  error.fields = fields; // erorile pe câmpuri (le folosim în laboratorul 10)
  return error;
}

export async function getReviews(productId) {
  const res = await fetch(`${REVIEWS_BASE}/api/products/${productId}/reviews`);
  if (!res.ok) throw await reviewError(res);
  return res.json();
}

export async function createReview(data) {
  const res = await fetch(`${REVIEWS_BASE}/api/reviews`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (!res.ok) throw await reviewError(res);
  return res.json();
}

export async function deleteReview(id) {
  const url = `${REVIEWS_BASE}/api/reviews/${id}`;
  const res = await fetch(url, { method: 'DELETE' });
  if (!res.ok) throw await reviewError(res);
}
```

De ce. `fetch` nu aruncă eroare la 400 sau 404, ci doar când serverul nu răspunde. De aceea verificăm `res.ok` (codurile 200–299) și transformăm mesajele din `ReviewController` într-o eroare JavaScript [2]. La DELETE (204, fără corp) nu apelăm `res.json()`.

5. Componenta ReviewSection

Creați folderul `frontend/app/src/components/` (dacă nu există) și în el fișierul `ReviewSection.jsx`. Componenta primește id-ul produsului prin *props* și ține în **stare** lista, încărcarea, eroarea și formularul.

```
import { useEffect, useState } from 'react';
import { getReviews, createReview, deleteReview } from '../api';

const EMPTY_FORM = { author: '', rating: '5', comment: '' };

export default function ReviewSection({ productId }) {
  const [reviews, setReviews] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [form, setForm] = useState(EMPTY_FORM);
  const [saving, setSaving] = useState(false);

  // Se execută la afișare și ori de câte ori se schimbă productId.
  useEffect(() => {
    setLoading(true);
    setError('');
    getReviews(productId)
      .then(setReviews)
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, [productId]);

  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value });
  }

  async function handleSubmit(e) {
    e.preventDefault();
    setSaving(true);
    setError('');
    try {
      const created = await createReview({
        author: form.author,
        rating: Number(form.rating),
        comment: form.comment,
        productId: Number(productId),
      });
      setReviews((list) => [created, ...list]); // recenzia nouă apare prima
      setForm(EMPTY_FORM);
    } catch (err) {
      setError(err.message);
    } finally {
      setSaving(false);
    }
  }

  async function handleDelete(id) {
    if (!window.confirm('Sigur ștergeți această recenzie?')) return;
    try {
      await deleteReview(id);
      setReviews((list) => list.filter((r) => r.id !== id));
    } catch (err) {
      setError(err.message);
    }
  }
```

```
  return (
    <section className="reviews">
      <h2>Recenzii ({reviews.length})</h2>
      {loading && <p>Se încarcă recenziile...</p>}
      {error && <p className="error">{error}</p>}
      {!loading && reviews.length === 0 && <p>Nu există încă recenzii.</p>}

      <ul className="review-list">
        {reviews.map((r) => (
          <li key={r.id}>
            <strong>{r.author}</strong>{' '}
            <span className="stars" role="img"
              aria-label={`${r.rating} din 5 stele`}>
              {'★'.repeat(r.rating)}{'☆'.repeat(5 - r.rating)}
            </span>
            <small> · {r.createdAt}</small>
            {r.comment && <p>{r.comment}</p>}
            <button type="button" onClick={() => handleDelete(r.id)}>Șterge</button>
          </li>
        ))}
      </ul>

      <form onSubmit={handleSubmit}>
        <h3>Adăugați o recenzie</h3>
        <label>
          Numele vostru
          <input name="author" value={form.author} onChange={handleChange}
            required />
        </label>
        <label>
          Nota
          <select name="rating" value={form.rating} onChange={handleChange}>
            {[5, 4, 3, 2, 1].map((n) => <option key={n} value={n}>{n}</option>)}
          </select>
        </label>

        <label>
          Comentariu (opțional)
          <textarea name="comment" value={form.comment} onChange={handleChange}
            rows={3} />
        </label>
        <button type="submit" disabled={saving}>
          {saving ? 'Se trimite...' : 'Trimite recenzia'}
        </button>
      </form>
    </section>
  );
}
```

6. Afișarea pe pagina produsului

1. În `pages/ProductDetailsPage.jsx`, găsiți variabila cu id-ul produsului din adresă (de obicei `id`, din `useParams()`).
2. Adăugați importul lângă celelalte importuri, la începutul fișierului:

   ```
   import ReviewSection from '../components/ReviewSection';
   ```

3. În JSX-ul returnat, sub detaliile produsului (înainte de ultima etichetă de închidere), adăugați componenta, cu variabila găsită la pasul 1:

```
<ReviewSection productId={id} />
```

4. Salvați și deschideți http://localhost:5173/produse/1. Adăugați o recenzie, apoi ștergeți-o.

Rezultat. Recenziile apar cu stele. Recenzia trimisă apare prima, iar formularul se golește; **Șterge** cere confirmare. După **F5**, datele rămân: sunt în PostgreSQL.

Verificarea în DevTools și a stării de eroare

- **F12** → **Network** → filtrul **Fetch/XHR**. Adăugați o recenzie: apar `OPTIONS` (*preflight*) și `POST reviews` cu **201**; la **Payload** vedeți JSON-ul trimis, la **Response** pe cel primit.
- Opriți backend-ul (`docker compose -f backend/compose.yaml stop`) și reîncărcați pagina. Componenta afișează mesajul de eroare în loc să „înghețe”. Porniți-l la loc cu `up -d`.

7. Salvarea lucrului și oprirea

```
git add .
git commit -m "Laboratorul 09: recenziile in pagina produsului (ReviewSection)"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

8. Exerciții

1. Afișați deasupra listei nota medie, cu o zecimală (de exemplu „Nota medie: 4.3 din 5”). Indiciu: `reviews.reduce(...)` și `toFixed(1)`; aveți grijă la lista goală.
2. După o trimitere reușită, afișați 3 secunde mesajul „Mulțumim pentru recenzie!” (o stare nouă și `setTimeout`).
3. ★ (opțional) Stilizați secțiunea în `App.css` (clasele `reviews`, `review-list`, `stars`, `error`), cu un contrast suficient între text și fundal.
4. ★ (opțional) Adăugați un buton **Editează**, care încarcă recenzia în formular și o trimite cu `PUT /api/reviews/{id}` (o funcție nouă `updateReview` în `api.js`).

9. Ce arătați la final

- adăugarea și ștergerea unei recenzii, cu DevTools → Network deschis (POST 201, OPTIONS explicat);
- commit-ul „Laboratorul 09” pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – JavaScript Fetch API:** https://www.w3schools.com/js/js_api_fetch.asp
- **W3Schools – React (formulare, useEffect):** https://www.w3schools.com/react/
- **React, în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PLZuysB_DjFARX5kWZGu8hvBbmw-3VWd8w
- **MDN – Cross-Origin Resource Sharing (CORS):** https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] MDN Web Docs, „Using the Fetch API”. developer.mozilla.org

[3] React, „Synchronizing with Effects” și „Updating Arrays in State”, documentație oficială. react.dev/learn

[4] MDN Web Docs, „Cross-Origin Resource Sharing (CORS)”. developer.mozilla.org

[5] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.